AI Laboratory

Bayesian Networks and Autoregressive Language Models

Krrish Bagaria

September 30, 2026

Contents

1 Objective **5**

2 Dataset **5**

3 Part I: From Probability to Language **6**

3.1 Question 1 6

4 Part II: Bayesian Network for Text **6**

4.1 Question 2 6

5 Part III: First-Order Conditional Probability Table **7**

5.1 Question 3 7

5.1.1 Context: the 7

5.1.2 Context: cat 8

5.1.3 Context: dog 8

5.1.4 Context: sat 8

5.1.5 Context: ran 8

6 Implementation of the First-Order Model **8**

6.1 Question 4 9

6.2 Question 5 9

6.3 Question 6 9

6.4 Question 7 10

7 Probability Model Testing **10**

7.1 Question 8 10

7.2 First-Order Normalisation Results 11

8 Next-Word Prediction **11**

8.1 Question 9 11

9 First-Order Text Generation **12**

9.1 20 Sampled First-Order Sentences 12

10 Deterministic vs Probabilistic Generation **13**

10.1 Question 10 13

11 Part XI: Second-Order Bayesian Network **14**

11.1 Question 11 14

11.1.1 1. Graph Structure 14

11.1.2 2. Conditional Probability Table 15

11.1.3 3. Amount of Context 15

12 Second-Order Conditional Probability Tables **15**

12.1 Context: `<START>, the` 15

12.2 Context: `the, cat` 15

12.3 Context: `the, dog` 16

12.4 Context: `cat, sat` 16

12.5 Context: `dog, sat` 16

12.6 Context: `cat, ran` 16

12.7 Context: `dog, ran` 16

12.8 Context: `sat, on` 16

12.9 Context: `on, the` 16

12.10 Context: `ran, to` 17

13 Second-Order Probability Testing **17**

14 Second-Order Text Generation **17**

14.1 Greedy Generation 17

14.2 Sampling Generation 18

15 Comparison of First-Order and Second-Order Models **18**

15.1 Question 12 18

16 Qualitative Comparison **19**

17 Connection to Modern Autoregressive Language Models **20**

17.1 Question 13 20

18 Modern Autoregressive Language Models **21**

19 Reflection on the Role of the LLM **21**

19.1 Question 14 21

20 LLM Usage and Validation **22**

20.1 Example of an LLM-Generated Error That Was Corrected 22

21 Final Observations **23**

22 Conclusion **24**

1 Objective

The objective of this laboratory is to understand the relationship between Bayesian networks, conditional probability distributions, and autoregressive language models.

A language model estimates the probability of the next token given previous tokens. In this laboratory, this idea is implemented using simple first-order and second-order Markov models.

The laboratory investigates:

- the chain-rule decomposition of a sequence probability;
- the representation of a language model as a Bayesian network;
- construction of conditional probability tables from data;
- next-word prediction;
- probabilistic text generation;
- the effect of increasing the amount of context;
- validation of an LLM-generated implementation.

The two models implemented are:

$$P(X_t \mid X_{t-1}) \tag{1}$$

for the first-order model, and

$$P(X_t \mid X_{t-2}, X_{t-1}) \tag{2}$$

for the second-order model.

2 Dataset

The following six sentences were used as the training dataset:

1. the cat sat on the mat
2. the cat sat on the rug
3. the dog sat on the mat
4. the dog ran to the park
5. the cat ran to the park
6. the dog sat on the rug

All text was converted to lowercase and each word was treated as a token.

The special tokens `<START>` and `<END>` were added to mark the beginning and end of each sentence.

For the first-order model, a sentence was represented as:

```
<START> the cat sat on the mat <END>
```

For the second-order model, two start tokens were used so that two previous tokens were available when predicting the first real word:

```
<START> <START> the cat sat on the mat <END>
```

3 Part I: From Probability to Language

3.1 Question 1

Why is the chain-rule decomposition useful for generating text?

The chain rule decomposes the probability of an entire sequence into a sequence of conditional probabilities:

$$P(x_1, x_2, \ldots, x_T) = P(x_1) \prod_{t=2}^{T} P(x_t \mid x_1, \ldots, x_{t-1}). \tag{3}$$

This is useful for text generation because the model does not need to generate the entire sentence at once. Instead, it can generate one token at a time.

After generating a token, that token can be used as context for predicting the next token. The process is therefore:

$$\text{predict token} \rightarrow \text{append token} \rightarrow \text{predict next token} \rightarrow \cdots$$

The complete sentence probability is obtained by multiplying the conditional probabilities of the individual tokens.

4 Part II: Bayesian Network for Text

4.1 Question 2

What independence assumption is being made by the first-order network?

The first-order model assumes that the next token depends only on the immediately preceding token.

Therefore,

$$P(X_t \mid X_1, \ldots, X_{t-1}) \approx P(X_t \mid X_{t-1}). \tag{4}$$

In other words, once X_{t-1} is known, the earlier tokens do not provide additional information for predicting X_t according to this model.

The corresponding Bayesian-network structure is:

$$X_1 \to X_2 \to X_3 \to \cdots \to X_T.$$

This is a strong simplifying assumption and is the main limitation of the first-order model.

5 Part III: First-Order Conditional Probability Table

The first-order model estimates:

$$P(w_j \mid w_i) = \frac{C(w_i, w_j)}{\sum_k C(w_i, w_k)}, \tag{5}$$

where $C(w_i, w_j)$ represents the number of times w_j occurs immediately after w_i in the training data.

5.1 Question 3

Construct the conditional probability distribution P(next word | current word) for the required words.

5.1.1 Context: the

The observed transitions from `the` are:

Next word	Probability
cat	0.2500
mat	0.1667
rug	0.1667
dog	0.2500
park	0.1667

The probabilities sum to:

$$0.25 + 0.1667 + 0.1667 + 0.25 + 0.1667 \approx 1.$$

5.1.2 Context: cat

Next word	Probability
sat	0.6667
ran	0.3333

5.1.3 Context: dog

Next word	Probability
sat	0.6667
ran	0.3333

5.1.4 Context: sat

Next word	Probability
on	1.0000

5.1.5 Context: ran

Next word	Probability
to	1.0000

All transitions not observed in the training data have probability zero. For example,

$$P(ran \mid sat) = 0$$

because “ran” was never observed immediately after “sat”.

6 Implementation of the First-Order Model

The first-order model stores transition counts using a nested dictionary. For example,

```
transition_counts["the"]["cat"] = 3
```

means that “cat” occurred three times immediately after “the”.

The conditional probability is then calculated by dividing each transition count by the total number of transitions observed from the current word.

6.1 Question 4

Where in the program are the transition counts stored?

The transition counts are stored in the dictionary:

`self.transition_counts`

It is a nested dictionary where the first key is the current word and the second key is the next word.

For example:

`self.transition_counts["the"]["cat"]`

stores the number of times "cat" follows "the".

6.2 Question 5

Where is $P(X_t \mid X_{t-1})$ computed?

The conditional probability is computed after all transition counts have been collected.

For a current word w_i and next word w_j, the implementation calculates

$$P(w_j \mid w_i) = \frac{C(w_i, w_j)}{\sum_k C(w_i, w_k)}. \tag{6}$$

The resulting values are stored in the dictionary:

`self.probabilities`

For example:

`self.probabilities["the"]["cat"]`

contains the value 0.25.

6.3 Question 6

How does the program choose the next word?

The program supports two generation methods.

Greedy generation always selects the word with the highest conditional probability:

$$w^* = \arg\max_w P(w \mid w_{\text{previous}}). \tag{7}$$

Sampling generation randomly samples from the complete conditional probability distribution.

Therefore, a word with a higher probability is more likely to be selected, but lower-probability words can also be selected.

6.4 Question 7

What happens if the program encounters a word for which no transition has been observed?

If a word is not present in the probability table, there is no observed transition from that word.

The implementation returns `None` in this situation. During sentence generation, this causes the generation process to stop rather than inventing an unsupported probability distribution.

This behaviour prevents the program from assigning arbitrary probabilities to unseen contexts.

7 Probability Model Testing

For a valid conditional probability distribution, the probabilities of all possible next words for a given context must sum to one:

$$\sum_{v} P(v \mid w) = 1. \tag{8}$$

7.1 Question 8

If one of the totals is 0.87, what does this tell you about the implementation?

A total of 0.87 would indicate that the probability distribution has not been constructed correctly.

For a valid conditional probability table,

$$\sum_{v} P(v \mid w) = 1.$$

Therefore, a value of 0.87 would suggest an error such as:

- incorrect transition counts;
- an incorrect denominator;
- some transitions being omitted;
- probabilities not being normalised correctly.

The implementation therefore explicitly checks the sum of every conditional probability distribution.

7.2 First-Order Normalisation Results

The program produced the following results:

Context	Probability Sum
<START>	1.000000
the	1.000000
cat	1.000000
sat	1.000000
on	1.000000
mat	1.000000
rug	1.000000
dog	1.000000
ran	1.000000
to	1.000000
park	1.000000

Therefore, all first-order probability distributions passed the normalisation test.

8 Next-Word Prediction

The model was used to identify the most probable next word for five different contexts.

8.1 Question 9

Are the most probable predictions always the same as the words that you would personally expect? What does this tell you about the difference between a probability model and human linguistic expectations?

No. The most probable prediction is determined entirely by the statistics present in the training data and by the assumptions of the model.

For example, the first-order model predicts:

$$\arg\max_{w} P(w \mid the) = cat.$$

However, the model also assigns non-zero probability to “mat”, “rug”, “dog”, and “park” after “the”. Some of these transitions are not linguistically natural in the context of a complete sentence, but they occur as adjacent word pairs somewhere in the training dataset.

This demonstrates that a probability model is not necessarily representing human linguistic expectations. It is estimating the probability distribution specified by the data and model assumptions.

9 First-Order Text Generation

The model generates text by repeatedly predicting or sampling the next word.
The process is:

$$X_1 \sim P(X_1)$$

followed by

$$X_2 \sim P(X_2 \mid X_1),$$

then

$$X_3 \sim P(X_3 \mid X_2),$$

and so on until the `<END>` token is reached.

9.1 20 Sampled First-Order Sentences

The following sentences were generated by sampling from the first-order model:

1. the dog sat on the cat ran to the park
2. the dog sat on the dog sat on the park
3. the dog ran to the park
4. the dog ran to the park
5. the park
6. the park
7. the rug
8. the park
9. the park
10. the mat
11. the cat sat on the rug
12. the rug
13. the park
14. the rug
15. the cat sat on the dog sat on the cat ran to the mat

16. the mat
17. the rug
18. the dog sat on the park
19. the cat sat on the mat
20. the dog sat on the park

These results show that the first-order model can generate both valid sentences and unusual sequences.

This occurs because the model only considers one previous word and therefore does not retain enough context to maintain the structure of a complete sentence.

10 Deterministic vs Probabilistic Generation

10.1 Question 10

Compare greedy and sampling generation. Which mode produces more variation and why?

Greedy generation is deterministic. At every step it chooses the most probable next word.

The five greedy generations produced:

1. the cat sat on the cat sat on the cat sat on the cat sat on the cat sat on
2. the cat sat on the cat sat on the cat sat on the cat sat on the cat sat on
3. the cat sat on the cat sat on the cat sat on the cat sat on the cat sat on
4. the cat sat on the cat sat on the cat sat on the cat sat on the cat sat on
5. the cat sat on the cat sat on the cat sat on the cat sat on the cat sat on

All five outputs are identical.

In contrast, sampling produced different sentences, for example:

1. the rug
2. the rug
3. the park
4. the park
5. the rug

Therefore, sampling produces more variation.

The reason is that greedy generation always selects

$$\arg\max_{w} P(w \mid context),$$

while sampling allows lower-probability outcomes to be selected according to their probabilities.

The repetitive greedy output also demonstrates a limitation of the first-order model. Because it repeatedly chooses the locally most probable transition, it can enter repetitive cycles.

11 Part XI: Second-Order Bayesian Network

The first-order model assumes:

$$P(X_t \mid X_1, \ldots, X_{t-1}) \approx P(X_t \mid X_{t-1}). \tag{9}$$

The second-order model instead assumes:

$$P(X_t \mid X_1, \ldots, X_{t-1}) \approx P(X_t \mid X_{t-2}, X_{t-1}). \tag{10}$$

The corresponding Bayesian-network structure is:

$$X_{t-2} \rightarrow X_t \leftarrow X_{t-1}.$$

11.1 Question 11

How does the second-order model differ from the first-order model?

11.1.1 1. Graph Structure

The first-order model has a chain-like structure:

$$X_{t-1} \rightarrow X_t.$$

The second-order model has two incoming dependencies:

$$X_{t-2} \rightarrow X_t \leftarrow X_{t-1}.$$

Thus, the second-order model uses two previous tokens instead of one.

11.1.2 2. Conditional Probability Table

The first-order model stores:

$$P(X_t \mid X_{t-1}).$$

For example:

$$P(sat \mid cat) = 0.6667.$$

The second-order model stores:

$$P(X_t \mid X_{t-2}, X_{t-1}).$$

For example:

$$P(sat \mid the, cat) = 0.6667$$

and

$$P(ran \mid the, cat) = 0.3333.$$

11.1.3 3. Amount of Context

The first-order model has access to one previous token, whereas the second-order model has access to two previous tokens.

This additional context allows the model to distinguish between situations that would appear identical to the first-order model.

12 Second-Order Conditional Probability Tables

12.1 Context: `<START>, the`

Next word	Probability
cat	0.5000
dog	0.5000

12.2 Context: `the, cat`

Next word	Probability
sat	0.6667
ran	0.3333

12.3 Context: `the, dog`

Next word	Probability
sat	0.6667
ran	0.3333

12.4 Context: `cat, sat`

Next word	Probability
on	1.0000

12.5 Context: `dog, sat`

Next word	Probability
on	1.0000

12.6 Context: `cat, ran`

Next word	Probability
to	1.0000

12.7 Context: `dog, ran`

Next word	Probability
to	1.0000

12.8 Context: `sat, on`

Next word	Probability
the	1.0000

12.9 Context: `on, the`

Next word	Probability
mat	0.5000
rug	0.5000

12.10 Context: `ran, to`

Next word	Probability
the	1.0000

13 Second-Order Probability Testing

The second-order model was also tested using the condition

$$\sum_{v} P(v \mid w_{t-2}, w_{t-1}) = 1. \tag{11}$$

The following contexts all produced a total probability of 1.000000:

Context	Probability Sum
(<START>, <START>)	1.000000
(<START>, the)	1.000000
(the, cat)	1.000000
(cat, sat)	1.000000
(sat, on)	1.000000
(on, the)	1.000000
(the, mat)	1.000000
(the, rug)	1.000000
(the, dog)	1.000000
(dog, sat)	1.000000
(dog, ran)	1.000000
(ran, to)	1.000000
(to, the)	1.000000
(the, park)	1.000000
(cat, ran)	1.000000

Therefore, the second-order model also passed the probability-normalisation test.

14 Second-Order Text Generation

14.1 Greedy Generation

The second-order greedy model produced:

1. the cat sat on the mat
2. the cat sat on the mat
3. the cat sat on the mat

4. the cat sat on the mat
5. the cat sat on the mat

The deterministic behaviour is expected because greedy generation always chooses the most probable next token.

14.2 Sampling Generation

Five sampled second-order sentences were:

1. the cat sat on the mat
2. the cat sat on the mat
3. the cat ran to the park
4. the dog sat on the mat
5. the cat ran to the park

Compared with the first-order model, the second-order model generates sentences that more closely follow the structures present in the training data.

15 Comparison of First-Order and Second-Order Models

15.1 Question 12

Why does increasing the amount of context potentially improve prediction? Why can it simultaneously make the model harder to estimate from limited data?

Increasing the context gives the model more information about the sequence.

For example, the first-order model only knows:

$$P(X_t \mid X_{t-1}).$$

The second-order model knows:

$$P(X_t \mid X_{t-2}, X_{t-1}).$$

Therefore, the second-order model can distinguish contexts that have the same final word but different preceding words.

For example,

$$P(mat \mid on, the) = 0.5$$

and

$$P(rug \mid on, the) = 0.5.$$

The additional context can therefore improve the specificity of prediction.

However, increasing the context also increases the number of possible conditional contexts.

If the vocabulary contains V possible tokens, then a first-order table can contain up to

$$V^2$$

possible word-to-word combinations, whereas a second-order table can contain up to

$$V^3$$

possible combinations.

With a small training dataset, many of these combinations will never be observed. This results in zero-probability or unseen contexts and makes the conditional probability table sparser.

Thus, more context can improve predictive specificity, but it also requires more training data to estimate the probabilities reliably.

16 Qualitative Comparison

The first-order model has a smaller context and consequently makes weaker predictions.

For example, after observing "the", it assigns non-zero probability to:

$$cat,\ dog,\ mat,\ rug,\ park.$$

This results in generated sequences such as:

```
the dog sat on the dog sat on the park
```

and

```
the cat sat on the cat sat on the cat ...
```

These sequences demonstrate the limitations of using only one previous token.

The second-order model uses two previous tokens and therefore imposes stronger local constraints. Its sampled outputs included:

```
the cat sat on the mat
the cat ran to the park
the dog sat on the mat
```

which are all structures represented in the training data.

The second-order model therefore produced more contextually constrained outputs for this dataset.

However, this does not mean that a second-order model is equivalent to a modern large language model. It is still a very small fixed-context probabilistic model.

17 Connection to Modern Autoregressive Language Models

17.1 Question 13

Why is Approach B preferable when constructing an intelligent system?

Approach B specifies the intended probabilistic behaviour explicitly:

$$P(X_t \mid X_{t-1}) \tag{12}$$

estimated from transition counts and used for sampling-based generation.

This is preferable because it separates the specification of the model from its implementation.

The important steps are:

1. **Specify intended behaviour:** Clearly state what probability distribution the program should estimate.
2. **Understand the representation:** Identify what the variables, contexts, counts, and probability tables represent.
3. **Validate the implementation:** Check whether the generated code actually implements the specified probability model.
4. **Test probabilistic invariants:** For example, verify that all conditional probability distributions sum to one.
5. **Distinguish implementation from model:** A program can execute successfully while still implementing the wrong probabilistic model.

Therefore, a precise specification makes it possible to inspect and verify the resulting implementation rather than simply assuming that generated code is correct.

18 Modern Autoregressive Language Models

A modern autoregressive language model also estimates a conditional distribution of the form

$$P(X_t \mid X_1, \ldots, X_{t-1}). \tag{13}$$

The main difference is how this conditional distribution is represented and learned.

In the simple model developed in this laboratory, the distribution is represented explicitly using conditional probability tables and estimated using counting.

A modern language model typically uses a neural network to estimate the conditional distribution.

The conceptual comparison is:

	Simple BN Model	Modern Autoregressive LM
Representation	CPTs	Neural network
Context	Fixed window	Large learned context
Parameters	Explicit probabilities	Learned weights
Learning	Counting	Gradient-based training
Generation	Sampling	Sampling

Thus, the simple models in this laboratory are not comparable to modern LLMs in expressive power. However, both are based on the same fundamental probabilistic idea: predicting the next token conditioned on previous tokens.

19 Reflection on the Role of the LLM

19.1 Question 14

What did thinking of the language model as a Bayesian network give you?

Thinking of the language model as a Bayesian network provided several useful perspectives.

1. **Representation of dependencies:** The Bayesian network explicitly shows which previous tokens influence the current token.
2. **Factorisation of the joint distribution:** The network provides a direct factorisation of the joint probability into conditional probabilities.
3. **Interpretation of conditional probabilities:** The CPTs provide a direct interpretation of probabilities such as $P(sat \mid the, cat)$.
4. **Principled generation:** Once the conditional probability distributions are known, text can be generated by repeatedly sampling from them.

5. **Understanding independence assumptions:** The first-order model makes the assumption that earlier tokens do not provide additional information once the immediately preceding token is known.
6. **Understanding the effect of context:** Comparing first-order and second-order models demonstrates how adding context changes the conditional probability distribution.
7. **Testing the implementation:** The probabilistic formulation provides properties that can be tested, such as the requirement that each conditional distribution sums to one.

Therefore, the Bayesian-network interpretation connects the implementation directly to a precise probabilistic specification.

20 LLM Usage and Validation

The LLM was used as a programming assistant rather than as a replacement for understanding the probabilistic model.

The implementation was developed by first specifying the desired model:

$$P(X_t \mid X_{t-1}) \tag{14}$$

for the first-order case, followed by

$$P(X_t \mid X_{t-2}, X_{t-1}) \tag{15}$$

for the second-order case.

The generated implementation was then inspected and tested using the training data and the probability-normalisation property.

20.1 Example of an LLM-Generated Error That Was Corrected

An important issue was identified during testing of the first implementation.

Initially, the same dataset representation was used for both the first-order and second-order models. The second-order representation contained two `<START>` tokens:

```
<START> <START> the cat ...
```

This was appropriate for the second-order model, but it caused an issue for the first-order model.

The first-order model consequently learned a transition of the form

$$P(\texttt{<START>} \mid \texttt{<START>}) > 0.$$

As a result, greedy generation could repeatedly produce

```
<START> <START> <START> ...
```

instead of starting with "the".

This was identified by inspecting the transition counts and generated sentences.

The implementation was corrected by using:

```
<START> the cat ... <END>
```

for the first-order model and

```
<START> <START> the cat ... <END>
```

for the second-order model.

After this correction, the first-order transition table contained

$$P(the \mid \texttt{<START>}) = 1,$$

and generation started correctly.

This demonstrated why LLM-generated code should be inspected against the intended probabilistic model rather than accepted solely because the program runs successfully.

21 Final Observations

The laboratory demonstrated the progression

Probability → Bayesian Network → Autoregressive Model → Language Generation

The first-order model used one previous token as context and was therefore simple but prone to repetitive or incoherent sequences.

The second-order model used two previous tokens, giving it more information for prediction. This resulted in more constrained and generally more coherent generated sentences for the given dataset.

Both models successfully produced conditional probability tables whose probabilities summed to one for every observed context.

The experiment also demonstrated that greedy and sampling-based generation behave differently. Greedy generation is deterministic and repeatedly selects the highest-probability transition, whereas sampling introduces variation by drawing from the entire conditional probability distribution.

Finally, the laboratory demonstrated the importance of validating LLM-generated code. The discovered `<START>` token issue showed that a program can run without errors while still implementing an unintended probabilistic model. Checking transition counts, probability normalisation, and generated outputs provided practical tests of correctness.

22 Conclusion

This laboratory provided a simple implementation of autoregressive language modelling using Bayesian networks.

The first-order model estimated

$$P(X_t \mid X_{t-1}),$$

while the second-order model estimated

$$P(X_t \mid X_{t-2}, X_{t-1}).$$

The experiments showed that increasing the context provides additional information for prediction but also increases the number of possible contexts and therefore the amount of data required.

The Bayesian-network formulation made the dependencies and conditional probabilities explicit, while the generation process demonstrated how these probabilities can be repeatedly used to construct text.

The laboratory also demonstrated an important practical lesson in using LLMs for programming: the generated implementation must be compared against the intended mathematical specification and tested using properties that must hold for a valid probabilistic model.